

ESILV Smart Assistant — Technical Report

Document-Grounded Question Answering with Retrieval-Augmented Generation and Multi-Agent Orchestration

Lucas Perrier Zakaria Jalal

December 26, 2025

1 Introduction

Students and applicants at ESILV frequently require precise information regarding academic programs, administrative regulations, internship procedures, calendars, and institutional policies. This information is distributed across a large collection of heterogeneous documents, including PDFs, policy charters, calendars, and web pages. Manual exploration of these documents is time-consuming and error-prone, particularly when documents span multiple academic years or reuse similar terminology in different contexts.

While large language models (LLMs) provide a natural conversational interface for accessing information, they cannot be trusted to answer accurately without grounding. When queried directly, LLMs may hallucinate, infer outdated rules, or merge information across incompatible sources. The ESILV Smart Assistant addresses these limitations by adopting a Retrieval-Augmented Generation (RAG) architecture in which all answers are grounded in retrieved excerpts from official ESILV documents.

The project further explores a multi-agent architecture to separate responsibilities such as document-grounded question answering, structured lead capture, and deterministic FAQ handling. The system is implemented as a modular, testable codebase with clear integration points and is exposed through both a FastAPI + React stack and a Streamlit-based demonstration interface.

2 Problem Definition and Objectives

The core problem addressed by this project is enabling reliable, efficient, and traceable access to institutional knowledge stored in unstructured documents. Traditional keyword search often fails when the user’s phrasing does not align with document wording, while generic conversational agents may produce confident but incorrect responses.

The primary objective of the ESILV Smart Assistant is therefore to provide conversational question answering that:

- retrieves relevant evidence from official ESILV documents,
- constrains generation to retrieved content only,

- exposes sources for traceability,
- behaves predictably when information is missing or ambiguous.

In addition to question answering, the system supports document upload and reindexing, basic administrative monitoring, and optional lead capture through a dedicated agent.

3 Methodology

This project follows an engineering-driven methodology focused on correctness, traceability, and reproducibility rather than end-to-end model performance alone. The system is designed around the principle that reliable question answering in an academic or administrative context requires strong grounding in authoritative documents and predictable failure modes.

The methodology is structured around four main phases: corpus preparation, retrieval construction, agent-based reasoning, and controlled generation.

3.1 Corpus Preparation

All source documents are treated as authoritative inputs and stored unchanged in a dedicated raw data directory. The ingestion pipeline is designed to be deterministic and repeatable: given the same corpus and configuration, it produces the same retrieval index.

Document preparation follows a sequential process of loading, cleaning, and chunking. Cleaning is intentionally conservative to avoid altering semantic content, focusing instead on normalization and the removal of extraction artifacts. Chunking uses overlapping windows to preserve context across boundaries while keeping chunk sizes suitable for embedding models.

3.2 Vector-Based Retrieval

Semantic retrieval is implemented using dense vector embeddings and cosine similarity. Both documents and user queries are embedded into the same vector space using a sentence-transformer model. Retrieval quality is controlled through a fixed top- k selection and a minimum similarity threshold, ensuring that only sufficiently relevant passages are considered during generation.

The retrieval interface is treated as a stable contract, allowing the rest of the system to evolve independently of the underlying vector store implementation.

3.3 Agent-Oriented Query Handling

Instead of a single monolithic chatbot, the system adopts a multi-agent approach. Queries are first classified by an orchestrator, which selects the appropriate specialized agent. This separation allows different interaction modes—document-grounded question answering, structured data collection, and deterministic FAQ handling—to be implemented and evaluated independently.

Routing decisions prioritize safe defaults: when classification is ambiguous, queries are routed to the retrieval agent rather than speculative generation.

3.4 Grounded Generation

Answer generation is explicitly constrained by retrieved context. The language model is instructed to rely exclusively on provided excerpts and to acknowledge uncertainty when the context is insufficient. This design choice prioritizes correctness and transparency over conversational fluency.

Generation is treated as a replaceable component, allowing the system to remain functional even when the language model is unavailable during testing or evaluation.

4 System Architecture Overview

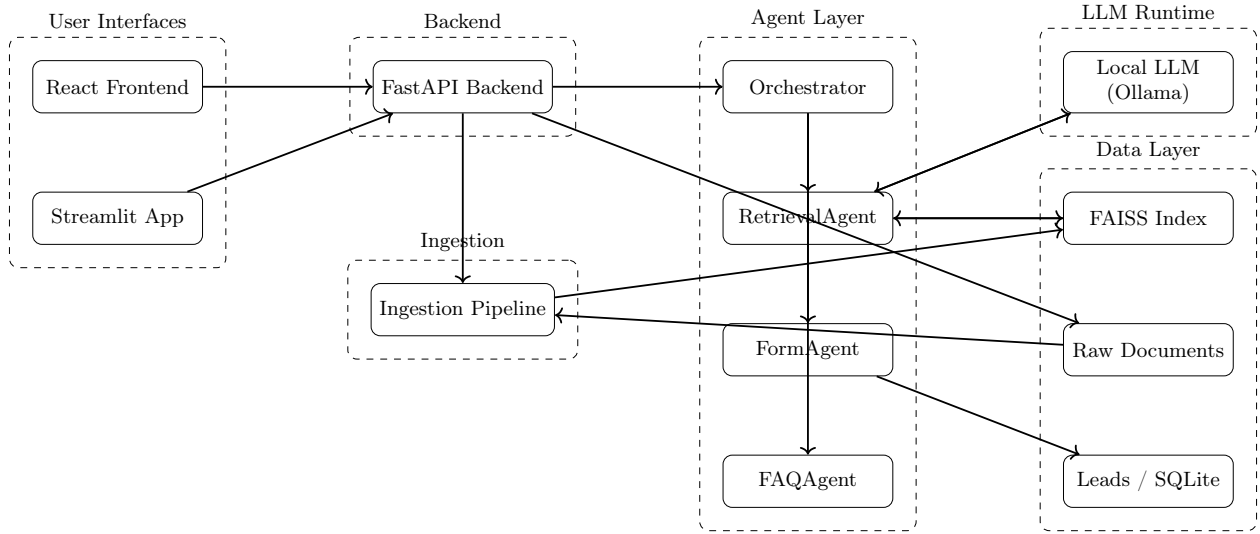


Figure 1: Architecture of the ESILV Smart Assistant. Queries are routed through a FastAPI backend to a multi-agent system. Document-grounded answers are generated via Retrieval-Augmented Generation using a FAISS vector index and a local LLM.

The system follows a standard Retrieval-Augmented Generation workflow augmented by a multi-agent orchestration layer. At runtime, a user submits a query through the web interface. The backend forwards this query to an orchestrator agent, which determines the appropriate handling strategy.

Most informational queries are handled by a RetrievalAgent, which retrieves relevant document chunks from a FAISS vector index, constructs a grounded prompt, and generates an answer using a local LLM accessed via Ollama. Queries that resemble structured form submissions are routed to a FormAgent, while a curated set of predictable questions can be answered deterministically by a FAQAgent.

Architecturally, the system is divided into three layers:

- **Ingestion layer** (`ingestion/`): document loading, cleaning, chunking, embedding, and vector index construction.
- **Agent layer** (`agents/`): orchestrator and specialized agents implementing distinct behaviors.

- **Serving and UI layer** (`backend/`, `frontend/`, and optional `app/`): FastAPI server, React frontend, and Streamlit demo.

This separation emphasizes modularity, testability, and stable integration points.

5 Repository Structure and Data Model

The repository is organized to reflect the architectural layers of the system. The `backend/` directory contains the FastAPI server implementing chat, upload, administration, reindexing, web ingestion, and optional student registry endpoints. The `frontend/` directory contains a React/Vite application that provides chat, upload, and admin interfaces. The `app/` directory contains a Streamlit-based demonstration interface.

Core agent logic resides in `agents/`, while the ingestion pipeline is implemented under `ingestion/`. Automated tests are located under `tests/`, and all persistent artifacts are stored under `data/`.

Within the data directory, the following artifacts are maintained:

- `data/raw/`: authoritative source documents used for ingestion.
- `data/vector_db/index.faiss` and `data/vector_db/index_metadata.json`: the authoritative retrieval index.
- `data/leads.json`: lead records persisted by the FormAgent.
- `data/students.db`: SQLite database used by administrative endpoints.
- `data/vector_index.json`: a lightweight stub index used only for upload and admin display.

A critical invariant of the system is that retrieval always uses the FAISS index under `data/vector_db/`. The stub index is never consulted during retrieval and must not be confused with the authoritative index.

6 Ingestion Pipeline

The ingestion pipeline transforms heterogeneous documents into a normalized set of text chunks and vector embeddings. Retrieval quality depends directly on the correctness and stability of this pipeline.

Document loading is implemented in `ingestion/loader.py`, which supports PDF, HTML, and plain text formats. Each loaded document is represented as a `Document` object containing extracted text, metadata (such as filename and extraction statistics), and a source path.

Extracted text is normalized by the cleaning stage (`ingestion/text_cleaning.py`), which applies whitespace and Unicode normalization, removes common extraction artifacts, and attempts to repair hyphenation across line breaks when detected.

The cleaned text is then split into overlapping chunks by the chunker (`ingestion/chunker.py`). The default configuration uses a chunk size of approximately 800 characters with an overlap of 150

characters. A recursive splitting strategy prioritizes semantic boundaries such as paragraphs before falling back to smaller units.

Each chunk is embedded using a sentence-transformers model (`all-MiniLM-L6-v2`), producing 384-dimensional vectors that are L2-normalized. Embeddings are stored in a FAISS `IndexFlatIP` index, while chunk texts and metadata are written to a sidecar JSON file aligned by index position.

The canonical ingestion command used throughout the project is:

```
python -m ingestion.pipeline -data-dir data/raw -output data/vector_db/index
```

7 Retrieval-Augmented Generation

Retrieval is performed through a stable vector store API:

```
ingestion.vector_store.search(query: str, top_k: int = 5) -> List[Tuple[str, dict]]
```

At runtime, the `RetrievalAgent` embeds the user query into the same vector space as the indexed chunks, retrieves the top- k most similar chunks, and applies a minimum similarity threshold (default value 0.3). Retrieved chunks are assembled into a context block used for generation.

Grounded generation is implemented in `BaseAgent.generate_response`. A system prompt enforces strict grounding constraints, instructing the model to rely exclusively on the provided context and to explicitly admit uncertainty when the context is insufficient. The default LLM is accessed via Ollama and can be configured through environment variables.

The `RetrievalAgent` exposes a stable interface:

```
RetrievalAgent.answer(query: str) -> str
```

Internally, responses may also include sources and action metadata, enabling consistent rendering across user interfaces.

8 Multi-Agent Orchestration

Rather than relying on a monolithic chatbot, the system employs multiple specialized agents coordinated by an orchestrator (`agents/orchestrator.py`). The orchestrator classifies incoming queries and dispatches them to the appropriate agent.

Classification is currently implemented using a lightweight LLM prompt. If the classifier output is ambiguous, the system defaults to retrieval. This approach offers flexibility at the cost of an additional LLM dependency.

The `FormAgent` (`agents/form_agent.py`) handles structured lead capture, incrementally collecting name, email, and interest fields and persisting results to `data/leads.json`. The `FAQAgent` (`agents/faq_agent.py`) provides deterministic answers for a curated set of frequently asked questions using token overlap scoring.

9 Backend and Frontend Integration

The FastAPI backend (`backend/main.py`) exposes REST endpoints for chat interactions, document upload, administration, reindexing, web ingestion, and a demonstration student registry backed by SQLite and SQLAlchemy.

The React frontend (`frontend/`) communicates with the backend via a typed API layer and provides separate views for chat, upload, and administration. An alternative Streamlit interface (`app/`) is provided for rapid experimentation and debugging.

10 Evaluation

The evaluation strategy focuses on two core dimensions of Retrieval-Augmented Generation systems: retrieval effectiveness and answer faithfulness. Rather than optimizing for stylistic quality, the evaluation prioritizes correctness, traceability, and predictable behavior.

10.1 Evaluation Objectives

The primary evaluation objectives are:

- to verify that relevant document passages are retrieved for representative queries,
- to ensure that generated answers are grounded exclusively in retrieved content,
- to confirm that the system refuses to answer when evidence is insufficient.

10.2 Qualitative Evaluation

Qualitative evaluation was performed using a representative subset of ESILV administrative documents, primarily PDF-based and spanning multiple academic years. Queries included factual questions (e.g., dates, rules, procedures) as well as procedural questions requiring multi-step explanations.

Observed behavior shows strong performance on factoid queries where relevant information is explicitly stated in the documents. Performance degrades for questions requiring synthesis across multiple document sections or when source documents suffer from poor text extraction quality.

Failure cases are typically attributable to one of three factors: insufficient lexical or semantic overlap between query and corpus, aggressive similarity thresholding that filters all candidates, or extraction artifacts in the source documents.

10.3 Retrieval-Focused Testing

The repository supports offline evaluation of retrieval independently from generation. Retrieval tests verify that the correct document chunks appear within the top- k results for known queries. This allows retrieval quality to be evaluated without relying on a language model.

Recommended retrieval metrics include Recall@ k and Mean Reciprocal Rank (MRR), computed on a small labeled set mapping queries to expected source documents.

10.4 Faithfulness and Safety

Answer faithfulness is evaluated by checking whether generated responses can be fully supported by retrieved excerpts. When no supporting evidence is present, the expected behavior is a refusal or explicit acknowledgment of uncertainty rather than speculative answering.

This behavior is enforced both through prompt constraints and through testing, where generation functions are mocked to ensure predictable outputs.

10.5 Limitations

The evaluation is primarily qualitative and corpus-specific. No large-scale benchmark dataset is used, and human judgment is required to assess answer correctness and citation relevance. Future work should formalize evaluation through labeled datasets and automated entailment checks.

11 Testing Strategy

Automated tests are organized under `tests/` and cover ingestion components, vector store behavior, agent logic, and orchestrator routing. A central design principle is that unit tests must not require a running Ollama daemon. LLM-dependent functionality is mocked, while integration tests that require Ollama can be explicitly marked and excluded from default test runs.

12 Future Work

While the ESILV Smart Assistant already provides a functional and well-structured Retrieval-Augmented Generation system, several extensions could significantly improve its robustness, scalability, and usability. These directions focus on addressing current limitations identified during development and evaluation, while preserving the system’s modular architecture.

12.1 Improved Query Routing

The current orchestrator relies on an LLM-based classifier to route queries between agents. Although flexible, this approach introduces additional latency and a dependency on LLM availability. A natural extension is a hybrid routing strategy in which lightweight heuristics are applied first. For example, queries containing email addresses or explicit personal information could be routed directly to the FormAgent, while exact or near-exact matches to curated questions could be handled by the FAQAgent. Only ambiguous cases would require LLM-based classification. This approach would improve reliability, reduce cost, and simplify testing.

12.2 Richer Citation and Provenance Tracking

At present, retrieved chunks include basic source metadata such as file names. Future work could propagate richer provenance information end-to-end, including page numbers, section titles, and chunk identifiers. This metadata could be surfaced in the user interface to allow users to inspect

original documents more easily. Stronger citation handling would further increase trust in the system and facilitate manual verification of answers.

12.3 Incremental and Continuous Indexing

The ingestion pipeline currently rebuilds the entire FAISS index when new documents are added. While acceptable for small corpora, this approach does not scale well. Incremental indexing strategies, such as appending new vectors or merging partial indices, would allow updates without full reprocessing. Coupled with file-level change detection, this would enable continuous ingestion and reduce downtime during updates.

12.4 Enhanced Document Extraction

Document extraction quality remains a major limiting factor, particularly for scanned PDFs, tables, and multi-column layouts. Future improvements include integrating optical character recognition (OCR) for scanned documents and adopting layout-aware PDF parsing tools to better preserve document structure. Improved extraction would directly enhance retrieval quality without modifying downstream components.

12.5 Evaluation Harness and Benchmarks

Current evaluation relies primarily on qualitative analysis and ad hoc testing. A dedicated evaluation harness could be introduced to systematically measure retrieval and generation quality. This would include a labeled set of representative questions with expected source documents, enabling computation of metrics such as Recall@k and Mean Reciprocal Rank. Automated faithfulness checks, potentially using entailment models, could further assess whether generated answers are supported by retrieved content.

12.6 User Feedback and Interaction Logging

Another promising direction is incorporating lightweight user feedback mechanisms, such as allowing users to flag incorrect or incomplete answers. Aggregated feedback could be used to identify problematic documents, refine chunking parameters, or adjust similarity thresholds. Interaction logs, collected in a privacy-conscious manner, would also support longitudinal analysis of system usage and performance.

12.7 Model and Deployment Flexibility

Finally, while the system currently targets local LLM deployment via Ollama, the architecture allows for alternative generation backends. Supporting multiple models or deployment modes (local versus remote) behind a common interface would increase portability and future-proof the system as model capabilities evolve.

13 Conclusion

The ESILV Smart Assistant demonstrates a robust and modular implementation of document-grounded question answering using Retrieval-Augmented Generation and multi-agent orchestration. By clearly separating ingestion, agent logic, and serving layers, and by emphasizing stable integration points and testability, the system provides a solid foundation for reliable student-facing access to institutional information. Incremental improvements in extraction quality, routing robustness, and evaluation methodology can further strengthen its reliability and applicability.